

Learning Observation Model for SLAM via Random Fourier Features in a Kernel Kalman Filter

HUBERT Bastien

DTIS, ONERA, Université Paris-Saclay

Palaiseau, France

bastien.hubert@onera.fr

DAHIA Karim

DTIS, ONERA, Université Paris-Saclay

Palaiseau, France

karim.dahia@onera.fr

MERLINGE Nicolas

DTIS, ONERA, Université Paris-Saclay

Palaiseau, France

nicolas.merlinge@onera.fr

GIREMUS Audrey

IMS, Université de Bordeaux, CNRS, Bordeaux INP

Talence, France

audrey.giremus@u-bordeaux.fr

Abstract—This paper addresses the problem of simultaneous localisation and mapping with an unknown observation model by formulating the navigation equations in reproducing kernel Hilbert spaces. This representation allows non-linearities to be handled through a Kalman-like recursion without explicit linearisations. Conjointly, the Hilbert space formulation is exploited to iteratively learn the observation model using a finite-dimensional random feature approximation, whose parameters are updated via recursive least squares minimisation.

The resulting algorithm preserves the prediction–update structure of Bayesian filtering while learning the environment model alongside the state estimation. Simulation results on a non-linear navigation scenario and comparison to a generic particle filter provided with a degraded observation function show that the proposed approach consistently estimates its state while reconstructing an accurate observation model.

I. INTRODUCTION

Simultaneous Localisation and Mapping (SLAM) is a fundamental problem in autonomous navigation, in which an agent must concurrently build a map of an unknown environment and estimate its own position relative to that map. This problem has been extensively studied over the past two decades [1, 2]. Traditional navigation approaches are typically described by probabilistic state-space models in which both the motion and observation processes are explicitly defined. Under Gaussian assumptions and moderate non-linearities, Kalman filtering techniques provide computationally efficient solutions [3]. In contrast, particle filter-based methods relax linearisation assumptions and improve robustness at the cost of potential sample degeneracy in high-dimensional spaces [4].

A key assumption underlying these formulations is the availability of an accurate observation model. In practice, however, such a function may involve complex, poorly parameterised, or unknown physical mechanisms. Visual SLAM [5] or landmark-based SLAM [6] algorithms rely on geometric features observable at a distance, reducing the need for an explicit environmental model. Conversely, in field-based SLAM such as radar-altimeter navigation [7] or magnetic navigation [8, 9], measurements correspond to local evaluations of an

unknown spatial field, making the observation function inherently difficult to specify analytically. This work focuses on this latter scenario.

We address the SLAM problem with an unknown observation function by embedding the navigation equations into reproducing kernel Hilbert spaces (RKHS) [10] and performing Bayesian inference using an Adaptive Kernel Kalman Filter (AKKF) [11]. Kernel embeddings allow non-linear transformations to be handled without explicit linearisations while preserving a Kalman-type recursive structure.

To obtain a computationally tractable algorithm, the observation model is approximated using Random Fourier Features (RFF) [12], which provide a finite-dimensional representation of the RKHS. The corresponding parameters are updated online via Recursive Least Squares (RLS) [13], enabling the map to be learned jointly with the state estimation while maintaining fixed per-step complexity.

Compared to particle filtering approaches, the proposed method avoids sample degeneracy and scales more favourably with state dimension. Unlike fully data-driven approaches such as neural networks [14] or Gaussian processes [15], it retains a fully recursive prediction–correction structure and does not require storing past observations. The resulting algorithm performs SLAM without requiring an explicit analytical observation model. Simulation results show competitive estimation accuracy with respect to particle-filter baselines.

Section II presents the general navigation problem under the RKHS paradigm, while section III focuses on approximating and learning the unknown observation function within the same assumptions. Section IV then provides numerical results of the proposed method on a simulated navigation scenario, and section V concludes the paper.

II. BAYESIAN FILTERING INSIDE A RKHS

Let us consider the following stochastic system formulated as a discrete state-space model (DSSM):

$$\begin{cases} x_k = F_k(x_{k-1}, \nu_k) \\ z_k = H_k(x_k) + \eta_k \end{cases} \quad (1)$$

where x_k represents the state of the system at time k , z_k is the observation of the environment by the system, and ν_k and η_k are random noises introduced to represent model inaccuracies.

In classical Bayesian filtering, the objective is to recursively estimate the posterior probability density function (PDF) of the state given all the observations available at time k . This is typically done via 2 equations:

- a propagation step, in which the prior PDF is computed using the Chapman-Kolmogorov equation:

$$p(x_k|z_{1:k-1}) = \int p(x_k|x_{k-1})p(x_{k-1}|z_{1:k-1})dx_{k-1} \quad (2)$$

- and a correction step, using Bayes' formula to derive the posterior PDF:

$$p(x_k|z_{1:k}) = \frac{p(z_k|x_k)p(x_k|z_{1:k-1})}{\int p(z_k|x)p(x|z_{1:k-1})dx} \quad (3)$$

While the integrals in (2) (3) have no analytical solution in general, particle filtering methods can be used to approximate the PDFs by Monte Carlo sampling. However, this comes at a high computational cost, and may suffer from sample degeneracy in situations where the optimal proposal density is difficult to sample from.

As an alternative, Kalman filtering approaches can be considered. However, when the system dynamics or observation functions are strongly non-linear and non-injective, they suffer from linearisation errors. To alleviate this issue, a recently proposed solution is to cast the system equations into a RKHS [10], where non-linear transformations can be handled implicitly. In this paper, we focus on kernel-based methods, which provide a linear representation of probability distributions in the feature space, thus enabling the use of Kalman filtering beyond standard linear and Gaussian assumptions.

A. RKHS and Kernel Mean Embedding

Formally, a RKHS $(\mathcal{H}, \langle \cdot | \cdot \rangle_{\mathcal{H}}, k)$ is a Hilbert space of functions from some set $\Omega \subset \mathbb{R}^d$ to $\mathbb{R}$ with a kernel $k : \Omega \times \Omega \rightarrow \mathbb{R}$ such that:

$$\forall f \in \mathcal{H}, \forall x \in \Omega, \langle f | k(x, \cdot) \rangle_{\mathcal{H}} = f(x) \quad (4)$$

This is called the reproducibility property of $\mathcal{H}$, and k is the corresponding (unique) reproducing kernel of $\mathcal{H}$. This property means that the evaluation operator $e_x : f \in \mathcal{H} \mapsto f(x)$ for some $x \in \Omega$ can be seen as the inner product between f and the implicit feature map $\phi(x) \triangleq k(x, \cdot) \in \mathcal{H}$. In particular, many situations only require the use of ϕ through inner products, in which case the actual computation of the feature map can be avoided using the kernel trick:

$$\langle \phi(x) | \phi(x') \rangle_{\mathcal{H}} = k(x, x') \quad (5)$$

For X a random variable on Ω , let:

$$\mu_X \triangleq \mathbb{E}_X(\phi(X)) = \int_{\Omega} \phi(x)p(x)dx \quad (6)$$

be its kernel mean embedding (KME). It follows from the transport theorem that:

$$\forall f \in \mathcal{H}, \mathbb{E}_X(f(X)) = \int_{\Omega} f(x)p(x)dx = \langle f | \mu_X \rangle_{\mathcal{H}} \quad (7)$$

Thanks to (7), any expectation of RKHS functions can be written as inner products with the mean embedding μ_X . Once μ_X is known, computing the expectation reduces to a linear evaluation in $\mathcal{H}$.

Similarly, joint distributions of a pair of random variables (X, Z) on two RKHS $(\mathcal{H}_x, \langle \cdot | \cdot \rangle_{\mathcal{H}_x}, k_x)$ and $(\mathcal{H}_z, \langle \cdot | \cdot \rangle_{\mathcal{H}_z}, k_z)$ can be embedded by the bilinear operator:

$$\begin{aligned} \mathcal{C}_{XZ} &\triangleq \mathbb{E}_{XZ}(\phi_x(X) \otimes \phi_z(Z)) \\ &= \int_{\Omega_x \times \Omega_z} \phi_x(x) \otimes \phi_z(z)p(x, z)dx dz \end{aligned} \quad (8)$$

where $\otimes$ represents the tensor product in the relevant product space. Conditional distributions can also be represented in RKHSs through conditional kernel mean embeddings. In particular, there exists a linear operator $\mathcal{C}_{X|Z}$ such that:

$$\mu_{X|z} = \mathcal{C}_{X|Z}\phi_z(z) \quad (9)$$

where $\mu_{X|z}$ denotes the embedding of the conditional distribution $p(x|z)$, and $\mathcal{C}_{X|Z}$ verifies:

$$\mathcal{C}_{X|Z} = \mathcal{C}_{XZ}\mathcal{C}_{ZZ}^{-1} \quad (10)$$

This provides a RKHS counterpart to the Bayesian correction step. In practice, the matrix inversion presented in (10) is modulated by a small regularisation coefficient for numerical stability purposes, such that we instead use:

$$\mathcal{C}_{X|Z} = \mathcal{C}_{XZ}(\mathcal{C}_{ZZ} + \lambda_z I)^{-1} \quad (11)$$

However, since the numerical computations of (6) and (8) are as difficult as that of (2), empirical Monte Carlo estimators are introduced instead. Formally, for two sets of N particles $\mathbf{x} = [x^1, \dots, x^N]^T \in \Omega_x^N$ and $\mathbf{z} = [z^1, \dots, z^N]^T \in \Omega_z^N$, let $\Phi_{\mathbf{x}} \triangleq [\phi_x(x^1), \dots, \phi_x(x^N)]$ and $\Phi_{\mathbf{z}} \triangleq [\phi_z(z^1), \dots, \phi_z(z^N)]$. The empirical KME and empirical joint operator are respectively defined as:

$$\hat{\mu}_X \triangleq \frac{1}{N} \sum_{i=1}^N \phi_x(x^i) \quad (12)$$

and:

$$\hat{\mathcal{C}}_{XZ} \triangleq \frac{1}{N} \sum_{i=1}^N \phi_x(x^i) \otimes \phi_z(z^i) = \frac{1}{N} \Phi_{\mathbf{x}} \Phi_{\mathbf{z}}^T \quad (13)$$

It comes from the Woodbury inversion formula that the conditional KME can be approximated as a linear combination of the feature map:

$$\mu_{X|z} \approx \hat{\mu}_{X|z} = \Phi_{\mathbf{x}} \underbrace{(\Phi_{\mathbf{z}}^T \Phi_{\mathbf{z}} + \lambda_z I)^{-1} \Phi_{\mathbf{z}}^T \phi_z(z)}_{\mathbf{w} = [w^1, \dots, w^N]^T \triangleq} \quad (14)$$

Using (4) and (7) finally yields:

$$\mathbb{E}_{X|z}(X) \approx \langle \text{id} | \Phi_{\mathbf{x}} \rangle_{\mathcal{H}_x} \mathbf{w} = \sum_{i=1}^N w^i x^i \quad (15)$$

where id denotes the identity function.

B. The Kernel Kalman Filter and its Adaptive Version

Introduced in [16], the Kernel Kalman Filter is able to compute the optimal Bayesian estimator by casting system (1) into two RKHS $(\mathcal{H}_x, \langle \cdot | \cdot \rangle_{\mathcal{H}_x}, k_x)$ and $(\mathcal{H}_z, \langle \cdot | \cdot \rangle_{\mathcal{H}_z}, k_z)$. While (1) is not assumed to be linear itself, the mean embedding methods presented above preserve the optimality properties of Kalman filtering in the RKHS representation. The Kernel Kalman Filter extensively uses the kernel trick to avoid the actual computation of the feature maps, relying only on vectorised evaluations of the relevant kernels.

Building from this algorithm, [11] introduces the Adaptive Kernel Kalman Filter (AKKF), which handles cases where the system's equations vary over time by recomputing the Gram matrices involved in the kernel Kalman filter at each time step. Algorithm 1 presents one iteration of AKKF, with the convention that $\tilde{\mathbf{x}}_0 = \mathbf{x}_0$ and $\mathbf{w} = [1/N, \dots, 1/N]$.

It should be noted that the algorithm from [16] introduces a matrix $O_k \triangleq (k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1}) + \lambda_x I)^{-1} k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1})$ which is omitted by AKKF. This is due to the fact that λ_x is assumed to be small, and so $O_k \approx I$ to the first order.

Algorithm 1 AKKF iteration

DSSM: prediction

$$\forall i \in [1, N], x_k^i \sim p(x_k | \tilde{x}_{k-1}^i) \\ \mathbf{x}_k \leftarrow [x_k^1, \dots, x_k^N]^T$$

RKHS: prediction

$$D_k \leftarrow (k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1}) + \lambda_x I)^{-1} k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1}) - I \\ P_{k|k-1} \leftarrow \tilde{P}_{k-1|k-1} + \frac{1}{N} D_k D_k^T$$

DSSM: correction

$$\forall i \in [1, N], z_k^i \sim p(z_k | x_k^i) \\ \mathbf{z}_k \leftarrow [z_k^1, \dots, z_k^N]^T$$

RKHS: correction

$$Q_k \leftarrow P_{k|k-1} (k_z(\mathbf{z}_k, \mathbf{z}_k) P_{k|k-1} + \lambda_z I)^{-1} \\ \mathbf{w}_k \leftarrow \tilde{\mathbf{w}}_{k-1} + Q_k (k_z(\mathbf{z}_k, \mathbf{z}_k) - k_z(\mathbf{z}_k, \mathbf{z}_k) \tilde{\mathbf{w}}_{k-1}) \\ P_{k|k} \leftarrow P_{k|k-1} - Q_k k_z(\mathbf{z}_k, \mathbf{z}_k) P_{k|k-1}$$

DSSM: state estimation

$$\hat{x}_k \leftarrow \sum_{i=1}^N w_k^i x_k^i \\ \hat{P}_k \leftarrow \sum_{i=1}^N w_k^i (x_k^i - \hat{x}_k)(x_k^i - \hat{x}_k)^T$$

DSSM: resampling

$$\forall i \in [1, N], \tilde{x}_k^i \sim \mathcal{N}(\hat{x}_k, \hat{P}_k) \\ \tilde{\mathbf{x}}_k \leftarrow [\tilde{x}_k^1, \dots, \tilde{x}_k^N]^T$$

RKHS: resampling

$$T_k \leftarrow (k_x(\tilde{\mathbf{x}}_k, \tilde{\mathbf{x}}_k) + \lambda_x I)^{-1} k_x(\tilde{\mathbf{x}}_k, \mathbf{x}_k) \\ \tilde{\mathbf{w}}_k \leftarrow T_k \mathbf{w}_k \\ \tilde{P}_{k|k} \leftarrow T_k P_{k|k} T_k^T$$

III. LEARNING THE OBSERVATION MODEL

A key limitation of algorithm 1 is that it requires sampling from the likelihood density $p(z_k | x_k)$, or equivalently evaluating H_k , which is assumed to be unknown in our application. Since the filtering problem is already formulated in a RKHS,

it is natural to model H_k within the same space. This section addresses the issue by introducing Random Fourier Features (RFF) [12, 17] to provide a finite-dimensional approximation within $\mathcal{H}_z$, and Recursive Least Squares (RLS) [13] as a means to recursively learn the approximation parameters.

A. Random Fourier Features

The Moore–Aronszajn theorem [18] establishes that a RKHS is the closure of the linear span of its feature maps:

$$\mathcal{H}_x = \overline{\text{span}}\{\phi_x(x), x \in \Omega_x\} \quad (16)$$

Since H_k is assumed to belong to $\mathcal{H}_x$ for the purpose of AKKF, it can thus be approximated arbitrarily well by a finite linear combinations of feature maps. In particular, given a sufficiently rich particle set, there exists $\tilde{\mathbf{a}} = [\tilde{a}^1, \dots, \tilde{a}^N]^T \in \mathbb{R}^N$ such that:

$$\forall x \in \Omega_x, H_k(x) \approx \sum_{i=1}^N \tilde{a}^i k_x(x_k^i, x) \quad (17)$$

Using the kernel trick, (17) can be rewritten as:

$$\forall x \in \Omega_x, H_k(x) \approx \langle \Phi | \phi_x(x) \rangle_{\mathcal{H}_x} \quad (18)$$

with:

$$\Phi \triangleq \sum_{i=1}^N \tilde{a}^i \phi_x(x_k^i) \in \mathcal{H}_x \quad (19)$$

Three main issues arise from this formulation:

- ϕ_x is either unknown or hard to evaluate in general, so the computation of Φ is equally challenging,
- $\mathcal{H}_x$ is typically of unbounded dimension, but RLS requires a finite representation of Φ to learn its parameters,
- Even with a finite formulation, $\langle \cdot | \cdot \rangle_{\mathcal{H}_x}$ depends on the structure of k_x , and its computation is non-trivial.

In practice, these difficulties make a direct RKHS implementation impossible. To overcome these limitations, it is necessary to obtain an explicit finite-dimensional approximation of k_x , which RFF provides.

Assuming the chosen reproducing kernel is stationary, there exists $c \in \mathbb{R}$ and $k' : \mathbb{R}^d \rightarrow \mathbb{R}$ with $k'(0) = 1$ such that $\forall (x, x') \in \Omega_x^2, k_x(x, x') = c k'(x - x')$. It follows from Bochner's theorem that:

$$k'(x - x') = \int_{\mathbb{R}^d} \exp(j\omega^T(x - x')) p(\omega) d\omega \\ = \int_{\mathbb{R}^d} \exp(j\omega^T x) \overline{\exp(j\omega^T x')} p(\omega) d\omega \quad (20) \\ = \mathbb{E}_\omega(\varphi_\omega(x)^T \varphi_\omega(x'))$$

where $p(\omega)$ is the spectral density of the kernel, and:

$$\varphi_\omega : x \in \Omega_x \mapsto \sqrt{2} \cos(\omega^T x + b), \quad b \sim \mathcal{U}([0, 2\pi]) \quad (21)$$

are the real-valued random Fourier features. This representation is obtained by exploiting the symmetry of the conjugate terms in the integral and the fact that k' is itself real-valued.

Since the integral in (20) has no explicit solution in general, we resort to Monte Carlo approximations once again by

sampling D frequencies $[\omega_1, \dots, \omega_D]$ from $p(\omega)$, and D phase offsets $[b_1, \dots, b_D]$ from $\mathcal{U}([0, 2\pi])$. By defining:

$$\varphi : x \in \Omega_x \mapsto \sqrt{\frac{2}{D}} \begin{bmatrix} \cos(\omega_1^T x + b_1) \\ \vdots \\ \cos(\omega_D^T x + b_D) \end{bmatrix} \quad (22)$$

it comes that:

$$k_x(x, x') \approx c \varphi(x)^T \varphi(x') \quad (23)$$

and thus:

$$\forall x \in \Omega_x, H_k(x) \approx \underbrace{\left(c \sum_{i=1}^N \tilde{a}^i \varphi(x_k^i) \right)^T}_{\mathbf{a} \triangleq} \varphi(x) \quad (24)$$

B. Recursive Least Squares

Written in the form of (24), H_k is approximated by a linear combination of known basis functions given by the random Fourier feature vector φ . The goal of this section is to learn the weight vector $\mathbf{a} \in \mathbb{R}^D$ given a sequence of training pairs $(\hat{x}_k, z_k)$, where $\hat{x}_k$ denotes the state estimate provided by the AKKF and z_k the associated real observation.

The learning problem up to time k is formulated as the minimisation of the following cost function:

$$J : \mathbf{a} \mapsto \sum_{l=1}^k \beta^{k-l} \|z_l - \mathbf{a}^T \varphi(\hat{x}_l)\|^2 + \beta^k \lambda_{RLS} \|\mathbf{a}\|^2 \quad (25)$$

where $\lambda_{RLS} \in \mathbb{R}_+$ and $\beta \in (0, 1]$ are some regularisation and forgetting parameters respectively.

As established by [13], RLS provides an efficient online solution to this problem: by computing the best linear unbiased estimator in a similar fashion to that of the Kalman correction, it updates the estimated weights using only the current data at each time step. Algorithm 2 gives a single iteration of the recursive process used by RLS to minimise this criterion, with the convention that $\mathbf{a}_0 = 0$, and $R_0 = I/\lambda_{RLS}$.

Algorithm 2 1 iteration of RLS

$$\begin{aligned} \mathbf{k}_k &\leftarrow R_{k-1} \varphi(\hat{x}_k) (\beta + \varphi(\hat{x}_k)^T R_{k-1} \varphi(\hat{x}_k))^{-1} \\ \mathbf{a}_k &\leftarrow \mathbf{a}_{k-1} + \mathbf{k}_k (z_k - \mathbf{a}_{k-1}^T \varphi(\hat{x}_k)) \\ R_k &\leftarrow \frac{1}{\beta} (I - \mathbf{k}_k \varphi(\hat{x}_k)^T) R_{k-1} \end{aligned}$$

C. RFF/RLS AKKF SLAM

Since the RLS update only requires a single pair $(\hat{x}_k, z_k)$ per iteration, the observation model can be learned in a fully recursive manner, naturally matching the time scale of the AKKF prediction–correction loop. In this article, we propose to combine both algorithms 1 and 2 to obtain a SLAM framework in which the observation model is adjusted jointly with state estimation, without requiring prior knowledge of the observation function. Each iteration of this algorithm consists of four main steps:

- a prediction step, in which a new set of particles is propagated through the state transition model,
- a correction step, where simulated observations are generated from the current approximation of the observation model and used for kernel-based filtering,
- an observation model update, performed by RLS using the real observation z_k and the current state estimate $\hat{x}_k$,
- a resampling step, ensuring consistency between the particle representation and the RKHS approximations.

Algorithm 3 presents an iteration of RFF/RLS AKKF SLAM. Note that the RLS update relies exclusively on the actual observation z_k , while the AKKF correction step uses simulated observations generated from the learned model.

Algorithm 3 1 iteration of RFF/RLS AKKF SLAM

DSSM: prediction

$$\begin{aligned} \forall i \in [1, N], x_k^i &\sim p(x_k | \tilde{x}_{k-1}^i) \\ \mathbf{x}_k &\leftarrow [x_k^1, \dots, x_k^N]^T \end{aligned}$$

RKHS: prediction

$$\begin{aligned} D_k &\leftarrow (k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1}) + \lambda_x I)^{-1} k_x(\tilde{\mathbf{x}}_{k-1}, \tilde{\mathbf{x}}_{k-1}) - I \\ P_{k|k-1} &\leftarrow \tilde{P}_{k-1|k-1} + \frac{1}{N} D_k D_k^T \end{aligned}$$

DSSM: correction

$$\begin{aligned} \forall i \in [1, N], \eta_k^i &\sim p(\eta_k) \\ \forall i \in [1, N], z_k^i &= \mathbf{a}_{k-1}^T \varphi(x_k^i) + \eta_k^i \\ \mathbf{z}_k &\leftarrow [z_k^1, \dots, z_k^N]^T \end{aligned}$$

RKHS: correction

$$\begin{aligned} Q_k &\leftarrow P_{k|k-1} (k_z(\mathbf{z}_k, \mathbf{z}_k) P_{k|k-1} + \lambda_z I)^{-1} \\ \mathbf{w}_k &\leftarrow \tilde{\mathbf{w}}_{k-1} + Q_k (k_z(\mathbf{z}_k, z_k) - k_z(\mathbf{z}_k, \mathbf{z}_k) \tilde{\mathbf{w}}_{k-1}) \\ P_{k|k} &\leftarrow P_{k|k-1} - Q_k k_z(\mathbf{z}_k, \mathbf{z}_k) P_{k|k-1} \end{aligned}$$

DSSM: state estimation

$$\begin{aligned} \hat{x}_k &\leftarrow \sum_{i=1}^N w_k^i x_k^i \\ \hat{P}_k &\leftarrow \sum_{i=1}^N w_k^i (x_k^i - \hat{x}_k)(x_k^i - \hat{x}_k)^T \end{aligned}$$

RFF/RLS: map update

$$\begin{aligned} \mathbf{k}_k &\leftarrow R_{k-1} \varphi(\hat{x}_k) (\beta + \varphi(\hat{x}_k)^T R_{k-1} \varphi(\hat{x}_k))^{-1} \\ \mathbf{a}_k &\leftarrow \mathbf{a}_{k-1} + \mathbf{k}_k (z_k - \mathbf{a}_{k-1}^T \varphi(\hat{x}_k)) \\ R_k &\leftarrow \frac{1}{\beta} (I - \mathbf{k}_k \varphi(\hat{x}_k)^T) R_{k-1} \end{aligned}$$

DSSM: resampling

$$\begin{aligned} \forall i \in [1, N], \tilde{x}_k^i &\sim \mathcal{N}(\hat{x}_k, \hat{P}_k) \\ \tilde{\mathbf{x}}_k &\leftarrow [\tilde{x}_k^1, \dots, \tilde{x}_k^N]^T \end{aligned}$$

RKHS: resampling

$$\begin{aligned} T_k &\leftarrow (k_x(\tilde{\mathbf{x}}_k, \tilde{\mathbf{x}}_k) + \lambda_x I)^{-1} k_x(\tilde{\mathbf{x}}_k, \mathbf{x}_k) \\ \tilde{\mathbf{w}}_k &\leftarrow T_k \mathbf{w}_k \\ \tilde{P}_{k|k} &\leftarrow T_k P_{k|k} T_k^T \end{aligned}$$

IV. APPLICATION AND NUMERICAL RESULTS

A. Simulation Scenario

In this section, we present the numerical application of algorithm 3, and its comparison to a degraded generic particle filter (PF) [4]. We consider a navigation scenario formulated as the DSSM introduced by (1) over $N_{iter} = 100$ steps, where

the evolution and observation functions are respectively chosen to be:

$$x_k = \begin{bmatrix} p_k^x \\ p_k^y \end{bmatrix} \triangleq \left(4.5 - 0.9 \frac{k}{N_{iter}} \right) \cdot \begin{bmatrix} \sin(2\pi \cdot 1.1 \frac{k}{N_{iter}} + 0.3) \\ \sin(2\pi \cdot 0.9 \frac{k}{N_{iter}} - 0.4) \end{bmatrix} \quad (26)$$

and:

$$H(x_k) \triangleq \sin(0.7p_k^x) + 0.5 \cos(1.2p_k^y) + 0.3 \sin(0.4(p_k^x + p_k^y)) \quad (27)$$

The process and measurement noises ν_k and η_k are modelled by Gaussian white noises with covariance matrices $Q = 0.01^2 I_2$ and $R = 0.01^2$ respectively. The initial state of the system is assumed to be shifted by a white Gaussian noise with covariance matrix $P_{init} = 0.1^2 I_2$. Figure 1 shows the ground-truth trajectory of the system over the observation field.

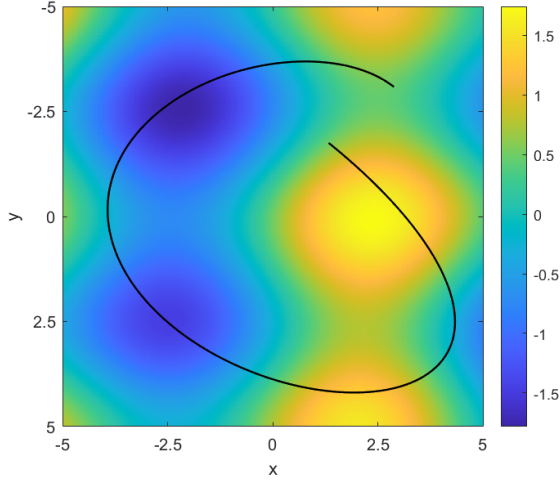


Fig. 1. Trajectory of the system over the observation field

Four filters are evaluated over $N_{MC} = 100$ Monte Carlo runs to solve the presented navigation problem:

- three PFs using a known but degraded observation function of the form:

$$H_\alpha(x) \triangleq (1 + \varepsilon(x)) \cdot H(x), \varepsilon(x) \sim \mathcal{N}(0, \alpha) \quad (28)$$

with degradation levels $\alpha \in \{0, 0.1, 0.5\}$,

- the RFF/RLS AKKF SLAM presented by algorithm 3.

Each filter uses 1000 particles. For the RKHS formulation, both kernels are chosen as Gaussian:

$$k(x, x') \triangleq \exp \left(-\frac{1}{2} \frac{\|x - x'\|_2^2}{1.5^2} \right) \quad (29)$$

and the regularisation parameters are set to $\lambda_x = \lambda_z = 0.1$. The RFF approximation is performed using $D = 500$ features, with frequencies sampled from a Gaussian distribution $p(\omega) \sim \mathcal{N}(0, 1.5)$. Finally, RLS is configured with a regularisation factor $\lambda_{RLS} = 0.01$ and a forgetting factor $\beta = 0.995$.

B. SLAM Performance

The root mean squared error (RMSE) of each filter at time step $k \in \llbracket 1, N_{iter} \rrbracket$ is defined as:

$$\text{RMSE}_x(k) \triangleq \sqrt{\frac{\sum_{m=1}^{N_{MC}} \|x_k - \hat{x}_{k,m}\|_2^2}{N_{MC}}} \quad (30)$$

where $\hat{x}_{k,m}$ denotes the estimated state computed by the m -th Monte-Carlo run at time k . Figure 2 gives the RMSE_x curves of all four tested filters.

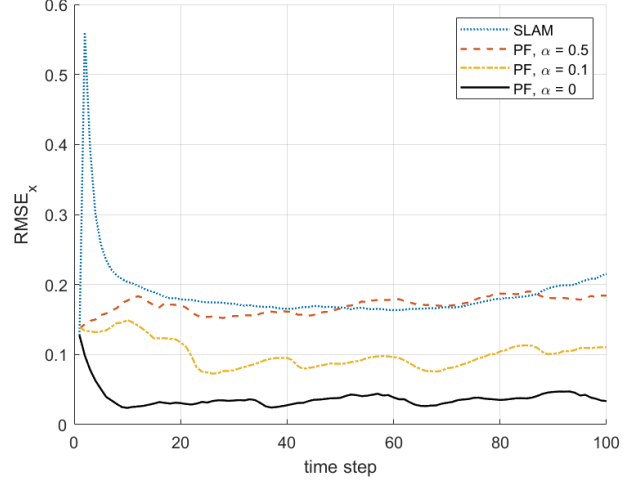


Fig. 2. RMSE_x as a function of time for the studied filters

Unsurprisingly, the RMSE_x for the PFs decrease with α , as the given observation function is increasingly precise. It must also be noted that with $\alpha = 0.5$, the PF is only able to maintain the initial estimation error without being able to converge to lower errors as $\alpha = 0.1$ and $\alpha = 0$ do. On the other hand, while the state RMSE for RFF/RLS AKKF SLAM only reaches that of the most degraded PF, it is able to reduce its high early estimation error from 0.56 at time step $k = 5$ down to 0.19 during the last 20 iterations. This is due to the fact that algorithm 3 has to learn the observation model, while the other filters have direct access to an already known, albeit degraded, observation function H_α . Table I computes the early stage maximum RMSE_x to average final RMSE_x for the four evaluated filters.

TABLE I
TRANSIENT TO ASYMPTOTIC RMSE_x COMPARISON
ACROSS THE TESTED FILTERS

	SLAM	PF, $\alpha = 0.5$	PF, $\alpha = 0.1$	PF, $\alpha = 0$
Maximum RMSE_x for $k \in \llbracket 1, 10 \rrbracket$	0.56	0.18	0.15	0.13
Mean RMSE_x for $k \in \llbracket 80, 100 \rrbracket$	0.19	0.18	0.11	0.04
Early maximum/late mean ratio	2.88	0.96	1.38	3.15

While the absolute RMSE_x remains high for algorithm 3, the transient to asymptotic ratio highlights a decrease of the

estimation error for RFF/RLS AKKF SLAM similar to that of the PF with access to the perfect observation model. This suggests that the iterative learning of the observation model proposed by our method is able to reduce the long-term error, even when the early RMSE_x is more pronounced.

Similarly to (30), the reconstruction error of the observation function is evaluated through the spatial RMSE:

$$\text{RMSE}_z : x \in \mathbb{R}^2 \mapsto \sqrt{\frac{\sum_{m=1}^{N_{MC}} \|H(x) - \hat{H}_m(x)\|_2^2}{N_{MC}}} \quad (31)$$

where $\hat{H}_m$ denotes to the final estimation of the map by the m -th run of algorithm 3 as described by (24). Figure 3 presents representative reconstruction of the observation function, while 4 shows the spatial distribution of RMSE_z . Its average is 0.21 over the $[-5, 5] \times [-5, 5]$ domain, reduced to 0.07 along the true trajectory.

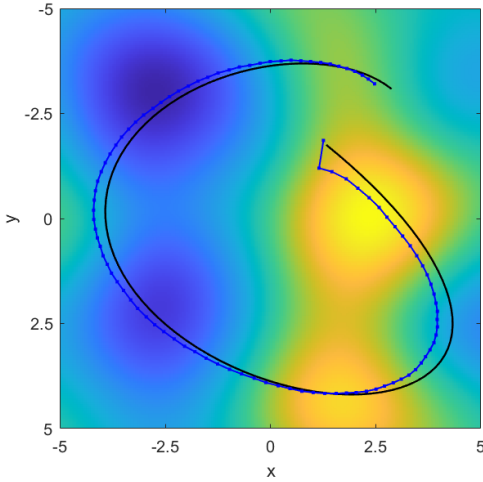


Fig. 3. An example of reconstructed observation function by algorithm 3 and its corresponding estimated trajectory in dotted blue

Four reconstruction error peaks can be observed in figure 4. Three of them are located on the exterior of the trajectory, and correspond to extrapolation errors that would likely be smaller if the state trajectory were to cover those regions. The fourth, comparatively smaller, peak is located near the inner portion of the trajectory, and should primarily be attributed to state estimation errors rather than intrinsic map regression errors. Since the RLS process depends explicitly on the estimated state $\hat{x}_k$, any localisation bias induces a spatial offset in the reconstructed observation field. Indeed, figure 3 clearly shows a shifted reconstruction compared to figure 1 around the centre of the trajectory.

V. CONCLUSION AND FUTURE WORK

In this paper, we introduced a new simultaneous localisation and mapping (SLAM) framework that combines the Adaptive Kernel Kalman Filter (AKKF) with Random Fourier Features (RFF) and Recursive Least Squares (RLS) to jointly estimate the system state and learn the observation model in a fully

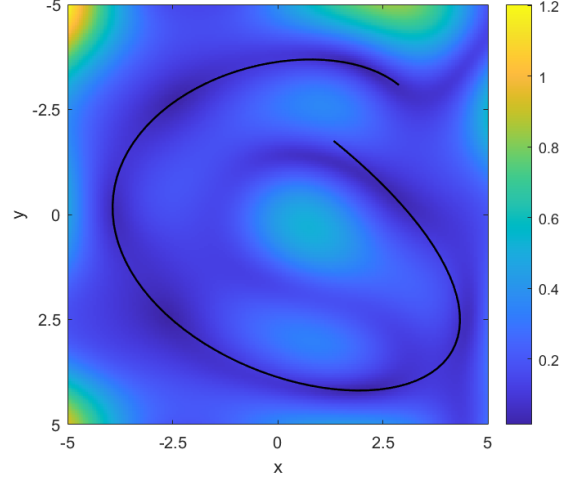


Fig. 4. Spatial distribution of the reconstruction error

recursive manner. By embedding both the state and observation spaces in Reproducing Kernel Hilbert Spaces (RKHS), the proposed approach extends the applicability of Kalman filtering to highly non-linear and non-injective systems, while enabling a recursive learning of the observation function.

Numerical results illustrate that, although the absence of a known observation function results in a larger initial estimation error, the proposed RFF/RLS AKKF SLAM progressively reduces both state and observation errors. The method achieves performance comparable to particle filtering approaches that assume partially known observation models. The spatial analysis of the reconstructed observation function further highlights the algorithm's capability to accurately learn the underlying map along the explored trajectory.

Future work will focus on explicitly integrating state uncertainty into the map regression, investigating the joint estimation of observation parameters with the state, and validating the method on real-world applications.

ACKNOWLEDGEMENT

The authors would like to thank Dr. Athénaïs Gautier for her helpful discussions and clarifications.

REFERENCES

- [1] H. Durrant-Whyte and T. Bailey. "Simultaneous Localization and Mapping (SLAM): Part I". In: *IEEE Robotics & Automation Magazine* 13.2 (2006), pp. 99–110.
- [2] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [3] S.J. Julier and J.K. Uhlmann. "A New Extension of the Kalman Filter to Nonlinear Systems". In: *Proceedings of AeroSense (The 11th International Symposium on Aerospace/Defense Sensing, Simulation and Controls)*. 1997, pp. 182–193.
- [4] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. "A Tutorial on Particle Filters for Online Nonlinear/Non-Gaussian Bayesian Tracking". In: *IEEE Transactions on Signal Processing* 50.2 (2002), pp. 174–188.

- [5] J. Engel, T. Schöps, and D. Cremers. “LSD-SLAM: Large-Scale Direct Monocular SLAM”. In: *Computer Vision – ECCV 2014, Lecture Notes in Computer Science*, vol. 8690. Springer, 2014, pp. 834–849.
- [6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit. “Fast-SLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem”. In: *Proceedings of the AAAI National Conference on Artificial Intelligence*. AAAI Press / The MIT Press, 2002, pp. 593–598.
- [7] C. Palmier, A. Giremus, P. Minvielle, C. P. Vacar, and G. Bourmaud. “Terrain-Aided SLAM with Limited-Size Reference Maps Using Gaussian Processes”. In: *Proceedings of the European Signal Processing Conference (EUSIPCO)*. 2023, pp. 1–5.
- [8] I. Vallivaara, J. Haverinen, and A. Kemppainen. “Simultaneous Localization and Mapping Using Ambient Magnetic Field”. In: *IEEE International Conference on Robotics and Automation*. 2011, pp. 1–6.
- [9] F. Viset, R. Helmons, and M. Kok. “An Extended Kalman Filter for Magnetic Field SLAM Using Gaussian Process Regression”. In: *Sensors* 22.8 (2022), pp. 1–19.
- [10] A. Gretton. “Introduction to RKHS, and Some Simple Kernel Algorithms”. In: *Adv. Top. Mach. Learn. Lecture Conducted from University College London* 16.5-3 (2013), p. 2.
- [11] M. Sun, M. E. Davies, I. K. Proudler, and J. R. Hopgood. “Adaptive Kernel Kalman Filter”. In: *IEEE Transactions on Signal Processing* 71 (2023), pp. 713–726.
- [12] A. Rahimi and B. Recht. “Weighted Sums of Random Kitchen Sinks: Replacing Minimization with Randomization in Learning”. In: *Advances in Neural Information Processing Systems* 21 (2008).
- [13] Z. Qin, B. Chen, and N. Zheng. “Random Fourier Feature Kernel Recursive Least Squares”. In: *2017 International Joint Conference on Neural Networks (IJCNN)* (2017), pp. 2881–2886.
- [14] M. Bloesch, J. Czarnowski, R. Clark, S. Leutenegger, and A. J. Davison. “CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2560–2568.
- [15] S. O’Callaghan and F. Ramos. “Gaussian Process Occupancy Maps”. In: *The International Journal of Robotics Research* 31.1 (2012), pp. 42–62.
- [16] G. H. W. Gebhardt, A. Kupcsik, and G. Neumann. “The Kernel Kalman Rule”. In: *Machine Learning* 108 (2019), pp. 2113–2157.
- [17] F. Liu, X. Huang, Y. Chen, and J. A. Suykens. “Random Features for Kernel Approximation: A Survey on Algorithms, Theory, and Beyond”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.10 (2021), pp. 7128–7148.
- [18] N. Aronszajn. “Theory of Reproducing Kernels”. In: *Transactions of the American mathematical society* 68.3 (1950), pp. 337–404.